

CS-1204

Object-Oriented Programming

(3+1)

Lecture

By
Engr. Muhammad Awais

Special Features in C++

Objectives

- By the end of this lecture, students will be able to:
 - ❑ Understand and apply friend functions to allow controlled access to private/protected members across classes.
 - ❑ Implement static data members and static member functions to share data and behavior across all objects of a class.
 - ❑ Define and use pure abstract functions to create abstract classes and enforce function overriding in derived classes.

Friend Functions

- **Friend functions** allow external functions or classes to access private and protected members of a class.
- Normally, private and protected members cannot be accessed from outside the class.
- A friend function provides controlled access to these members.
 - ❑ A **friend function** is **not a member** of the class.
 - ❑ It can access **private and protected members**.
 - ❑ It must be declared inside the class using the keyword friend

Syntax

```
class ClassName {  
    private:  
        int privateVar;  
  
    public:  
        ClassName(int x) : privateVar(x) {}  
  
        // Declaration of friend function  
        friend void showPrivate(ClassName obj);  
};  
  
// Definition of friend function  
void showPrivate(ClassName obj) {  
    std::cout << "Private variable: " << obj.privateVar << std::endl;  
}  
  
int main() {  
    ClassName obj(10);  
    showPrivate(obj); // Access private member  
    return 0;  
}
```

Example

```
D:\Data\Awais_Laptop\Univer X + v
Total area : 514
-----
Process exited after 0.2777 seconds with return value 0
Press any key to continue . . .
```

```
#include<iostream>
using namespace std;
class Rectangle; // forward declartion
class Circle{
private:
    int radius;
public:
    Circle(int r){
        this->radius = r;
    }
    //declartion of friend function
    friend double totalArea(Circle, Rectangle);
};

class Rectangle{
private:
    int length;
    int width;
public:
    Rectangle(int length, int width){
        this->length = length;
        this->width = width;
    }
    //declartion of friend function
    friend double totalArea(Circle c, Rectangle r);
};

//Definition of friend function who's friend of multiple classes
double totalArea(Circle c, Rectangle r){

    return (3.14 * c.radius * c.radius )+ (r.length * r.width);
}

int main(){

    Circle c(10);
    Rectangle r(20,10);
    // c.totalArea(c,r); //can not be used like this as total area is not a member of class.
    cout << "Total area : " << totalArea(c,r) << endl; //calling a member function same like normal function

    return 0;
}
```

Key Points About Friend Functions

- **Not a Member Function:** Friend functions are external to the class.
- **Declared with friend keyword:** Without friend, the function cannot access private/protected members.
- **Can Access Private/Protected Members:** That's the main purpose.
- **No this pointer:** Friend functions are not members, so they don't have a this pointer.
- **Can be Called Normally:** You call it like a regular function, not through an object.
- **Can be a Friend to Multiple Classes:** A friend function can access members of more than one class if declared in multiple classes.

Static Members

- In C++, **static members** are shared by all objects of a class. There are two types:
 - ❑ Static Data Members (Variables).
 - ❑ Static Member Functions
- **Key points:**
 - ❑ A static member belongs to the class itself, not to any particular object.
 - ❑ All objects of the class share the same static member.
 - ❑ Static members exist even if no objects of the class are created.
 - ❑ Can be accessed using the class name:
ClassName::staticMember.

Example of Static Data Members

```
#include<iostream>
using namespace std;

class A {
public:
    // Declaration
    static int count;
    A(){
        count++; // Increment for every object
    }
};
```

// Definition outside class

```
int A::count = 0;
```

```
int main(){
```

```
    A a;
```

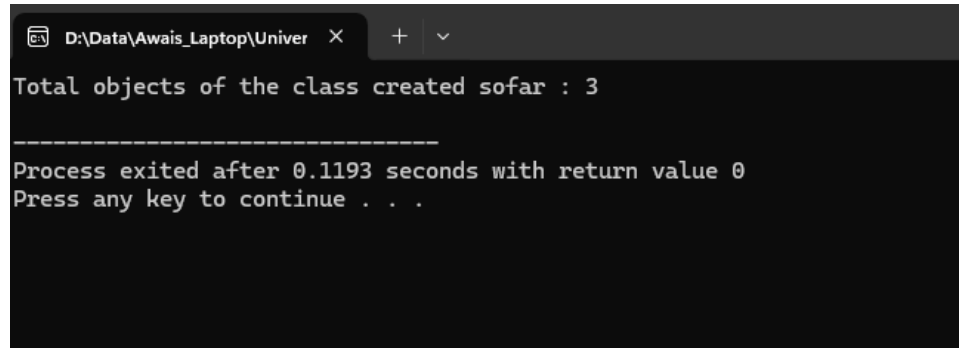
```
    A b;
```

```
    A c;
```

```
    cout << "Total objects of the class created sofar : " << A::count << endl; //calling a static member
```

```
    return 0;
```

```
}
```



The screenshot shows a terminal window with the following output:

```
D:\Data\Awais_Laptop\Univer  X  +  v
Total objects of the class created sofar : 3

-----
Process exited after 0.1193 seconds with return value 0
Press any key to continue . . .
```

Example of Static Members Functions

- Can only access static data member
- Do not have this pointer.
- Called using the class name.

```
#include<iostream>
using namespace std;

class A {
public:
    // Declaration
    static int count;
    A(){
        count++; // Increment for every object
    }

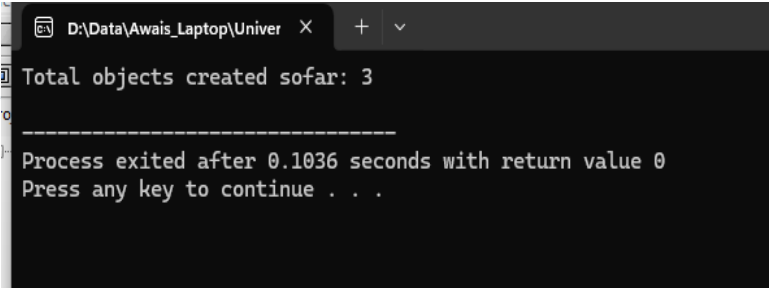
    static void showCount(){
        //can only access static members of the class.
        cout << "Total objects created sofar: " << count << endl;
    }
};

// Definition outside class
int A::count = 0;

int main(){

    A a;
    A b;
    A c;
    A::showCount(); //calling a static member functions

    return 0;
}
```



```
D:\Data\Awais_Laptop\Univer X + v
Total objects created sofar: 3
-----
Process exited after 0.1036 seconds with return value 0
Press any key to continue . . .
```

Pure Abstract Functions

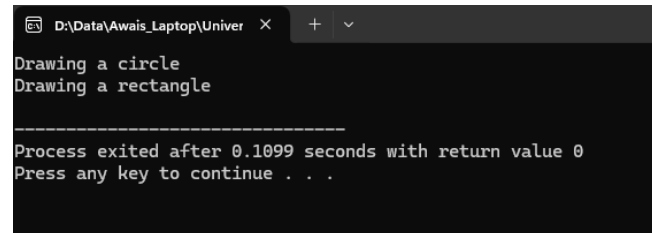
- A pure abstract function in C++ is a function that has no implementation in the base class and must be overridden by derived classes.
 - ❑ Used to create abstract classes.
 - ❑ Declared using = 0 syntax.
 - ❑ Makes the class abstract, meaning you cannot create objects of that class.
 - ❑ Derived class must override the function to become concrete.
 - ❑ You can have **multiple pure abstract functions** in a class.
 - ❑ Abstract classes can have **normal functions and data members** too.

Example of Pure Abstract Function

```
#include<iostream>
using namespace std;
class Shape{
public:
    virtual void draw() = 0; // pure abstract function
};
class Circle: public Shape{
public:
    void draw(){
        cout << "Drawing a circle" << endl;
    }
};

class Rectangle: public Shape{
public:
    void draw(){
        cout << "Drawing a rectangle" << endl;
    }
};

int main(){
    Circle c;
    Rectangle r;
    c.draw();
    r.draw();
    // Shape s; can not create an object of shape as it is abstract.
    // s.draw();
    return 0;
}
```



```
D:\Data\Awais_Laptop\Univer  X  +  v
Drawing a circle
Drawing a rectangle

-----
Process exited after 0.1099 seconds with return value 0
Press any key to continue . . .
```

Tasks - Friend Functions

Task 1 — Basic Friend Function

Create a class `Box` having:

- Private data member `length`
- A constructor to initialize `length`
- Create a friend function `displayLength()` that accesses and displays the private member.

Requirements

- Use `friend` keyword properly
- Call the friend function like a normal function
- Show output on screen

Tasks - Friend Functions

Task 2 — Mathematical Operation Using Friend Function

Create a class Distance with:

- Feet
- Inches

Create a friend function addDistance() that adds two distance objects and displays the result.

Requirements

- Use objects as parameters
- Perform proper addition logic

Tasks – Static Data Member

Task 1 — Bank Account System

Create a class BankAccount containing:

- Account holder name
- Balance

Use a static variable to store:

- Total number of bank accounts

Requirements

- Create multiple objects
- Display total accounts created

Tasks – Static Member Function

Task 1

Create a class Employee with:

- Employee ID
- Employee name

Use:

- Static variable for company name
- Static member function to display company information

Requirements

- Access static function using class name
- Demonstrate shared company name

Tasks – Pure Abstract Functions / Abstract Classes

Task 1

Create a hospital system where different staff members perform different duties.

Create abstract class HospitalStaff with:

- Staff name
- Pure virtual function performDuty()

Derived classes:

- Doctor
- Nurse
- Receptionist

```
Doctor is diagnosing patients  
Nurse is assisting patients  
Receptionist is managing appointments
```